'memory consistency'. By 'consistent', we denote when the updates made to memory locations by one processor become visible to a different processor in a multiprocessor system. It is quite easy to get confused between the terms 'coherence' and 'consistency' when discussing memory models for multiprocessor systems. Coherence ensures that the updates made to memory locations by one processor will eventually be made visible at other processors. However, it says nothing about when these updates will be made visible to other processors. On the other hand, the memory consistency model describes the order in which updates are made visible to remote processors. We defer the discussion of different memory consistency models to a future column, except to note that the interested reader can look up the paper "A tutorial on memory consistency models" by K. Ghattacharloo and S. Adve, available at www.hpl.hp.com/techreports/Compaq-DEC/WRL-95-7.pdf for a detailed description of these models.

For our current discussion, it is sufficient to know that a sequentially consistent memory model ensures that program order is obeyed for each thread of execution. This means that within a single thread of execution, the loads and stores happen in program order. A simple definition of the sequential consistency model is that the effect of memory operations (loads/stores) should take place as if they were executed in the program order. However, it does not impose any ordering among memory operations from different threads.

Now the question we ask ourselves is this: Do all modern processors enforce sequential memory consistency? Unfortunately, the answer is, "No". The reason is that enforcing sequential consistency has an impact on multiprocessor performance. Therefore, most processors support relaxed memory models, which do not enforce sequential consistency.

In such a case, how do we ensure the correctness of our mutual exclusion algorithms?

While enforcing sequential consistency for all memory operations would have an adverse effect on performance, most processors provide certain special instructions to enforce ordering among memory operations. These instructions are known as memory fences or memory barriers. A memory fence or memory barrier ensures that all outstanding memory operations are completed before proceeding past the barrier. However, the programmer needs to explicitly insert these memory fence instructions where they may be needed, placing a burden on the programmer to do the right thing. For example, in our *CombinedLock* algorithm, we need to have a memory fence right after setting the *flag* array element, and another fence after setting the *victim* variable. It is tempting for a newbie programmer to insert a memory fence after each read, to ensure correct ordering in implementing a mutual exclusion algorithm. However, the memory fence instructions are costly, so they need to be used sparingly. We will discuss memory fences in greater detail in our next column.

My 'must read' book for this month

I received a number of suggestions from readers of programming books they consider to be a must-read for every programmer. Thank you, and please keep the recommendations coming.

This month's book suggestion comes from S Ramakrishnan. You may be surprised to learn that it is not a computer science book at all. It is a little-known book titled 'How to Solve it' by George Polya, a Hungarian mathematician who wrote a number of books on mathematics, of which this is his most famous. The book contains different techniques and tips on problem solving.

So what makes this book interesting to a programmer? Well, every program we write is meant to solve some real-life problem. Before programmers dive into the actual coding of a programming problem, they should have a clear idea of the problem they are trying to solve, and how good their proposed solution is. Can it be improved; or, more importantly, is there already an existing solution? This book teaches you all this, and much more—so do make time to read it.

If you have a favourite book that you believe is a must-read for every programmer, please do send me a note with the book's name, and a short description of why you think it is useful. I will include a mention of it in this column. It will help many readers who want to improve their coding skills.

This month's takeaway problems

We have three takeaway questions for this month:

1. We discussed Petersen's algorithm in detail in this column. However, our discussion concerned only two threads. Can you generalise the algorithm for N threads? Do send me the pseudo-code for the generalised N threads case, for the Lock and Unlock routines.
2. We mentioned that enforcing sequential consistency models has an impact on multiprocessor performance. Can you explain why?
3. Do you need to employ any synchronisation primitives inside kernel code in uniprocessor systems? If yes, explain why? If not, why not?

Please do send me your answers to this month's questions along with your book recommendations. If you have any favourite programming puzzles that you would like to discuss on this forum, do send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming, and wish you all the very best! END

About Sandya Mannarswamy

The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit. Her areas of interest include virtualisation technologies and software development tools. [If you are preparing for computer science programming interviews, you may find it useful to visit Sandya's programming interviews discussion group 'Computer Science Interview Training (India)' on LinkedIn *(www.linkedin.com).*]